驾驭AI Coding：一份面向团队的Harness Engineering落地规范

原创 腾讯程序员 腾讯技术工程 2026年7月17日 02:36 广东 1466人



作者：atreusliu

> 交付代码的成本已经接近免费了，但交付好代码的成本依然很高。Harness Engineering 做的事情，就是把"好代码"的标准写进系统里，让 AI 在约束下自己干活。

为什么每位成员都必须遵循这套规范？

AI Coding 工具正在重塑软件开发的方式。当团队中每个人都能用 AI 快速生成代码时，真正拉开差距的不再是"谁写得快"，而是"谁写得好、谁写得稳、谁写得可维护"。

这套规范不是束缚，而是团队的共同语言和质量底线：

- **对个人**：它帮你建立正确的 AI 协作习惯，避免踩坑返工，让 AI 真正成为你的生产力倍增器
- **对团队**：它确保每个人产出的代码风格一致、架构统一、可审查可维护，降低协作摩擦
- **对项目**：它把质量标准固化到工具链中，让项目不会因为人员变动而失控

本文档的定位：

- 第一部分（一、二章）回答"为什么"和"是什么"：阐述 Harness Engineering 的核心理念和 AI Coding 一体化架构设计
- 第二部分（三~九章）回答"怎么做"：提供分阶段实施路线图、具体配置步骤、日常开发 SOP、反模式总结，以及基于我们构造的 `harness-audit` Skill 的自动化合规性自检
- 第三部分（十章）总结

关于阅读重点的一点说明：

MCP、Skills、Rules、SDD、知识库这些概念，网上和司内已经有大量入门文章讲过"它们是什么"，本文不再花篇幅重复这些老生常谈的定义。本文真正想讲清楚的，是另外两件事：

- **第一，它们在 Harness 体系中的功能定位。**同样是 MCP 和 Skills，单独看每一个工具都不难理解，但放进 Harness 的 6 大支柱里，各自承担什么角色、解决什么层面的问题、在 AI 工作流的哪个环节发挥作用，这才是决定团队能不能用好它们的关键。

- **第二，它们在实际开发场景中如何相互配合。**MCP 提供数据通道、Skills 封装领域经验、知识库注入业务上下文、Rules 划定行为边界——这几样工具不是孤立存在的，真正的威力在于组合使用。本文会结合具体场景（需求开发、Bug 修复、Code Review 等）讲清楚它们怎么协同工作。

其实我们日常开发中，已经或多或少在用 Harness 的思路了，只是缺一个系统化的框架把这些工具的使用方式和用法统一起来;

如果想捋清这些工具背后的设计逻辑，知道在自己的项目里该用哪一个、怎么搭配用、什么时候不该用，那这份规范就是为你准备的。

第一部分：理念与架构

一、核心理念：Harness Engineering（驾驭工程）

1.1 什么是 Harness Engineering?

2026 年 2 月，OpenAI 发了一篇文章《Harness Engineering: Leveraging Codex in an Agent-First World》。一个 3 人（后来扩到 7 人）的工程师团队，在完全禁止手写代码的条件下，用 AI Agent 在 5 个月内写了超过 100 万行代码，合并了 1,500 个 Pull Request，效率大概提升了 10 倍。

Harness 这个词来自马术，本意是"马具"——缰绳、马鞍、马镫。一匹没驯服的马力量很大，但你没法让它耕地、运货、上战场。AI 也一样:

> Agent = Model + Harness模型提供智能，Harness 让智能变成生产力。

LLM 本身没有状态、没有工具、没有记忆。Harness 层就是给模型装上"手脚和记忆"的工程基础设施。你写的所有代码、配的所有规则，都是 Harness 的一部分。



1.2 为什么需要 Harness？—— Vibe Coding 的三个致命问题

没有 Harness 约束的"氛围编码"（Vibe Coding），走的是一条 起步极快 → 中期混乱 → 后期崩盘 的路:

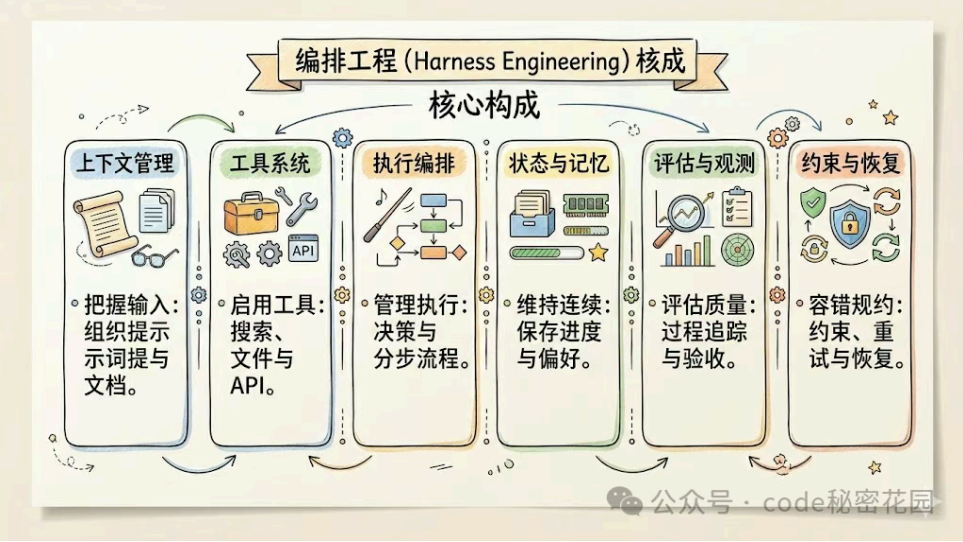
问题	现象	后果
架构混乱	Agent 喜欢走捷径，功能A用库X，功能B用库Y（哪怕X也能做），完全没有分层概念	一旦要换底层逻辑（比如换数据库），整个项目得大改
上下文雪崩	项目超过50个文件后，Agent 开始"忘事"——第1天用 `user_id`，第3天突然变成 `uid`	项目越大，Agent 越蠢，修一个 Bug 冒出两个新的
可维护性丧失	整个开发过程是黑盒，只有 Agent 知道代码怎么来的，人没参与思考	人想接手时，从头读几千行"垃圾代码"，还不如重写

Harness Engineering 就是来解决这些问题的：

- 安全边界：权限控制、审计日志、拒绝追踪
- 可观测性：Token 计数、成本追踪、决策日志
- 可靠性：重试机制、降级策略、确定性兜底
- 扩展性：工具生态、技能系统、多 Agent 协调

1.3 Harness 的 6 大支柱及其在 Coding 中的映射

Harness Engineering 把 Agent 的运行环境拆成 6 个支柱，每个支柱在我们的开发规范中都有对应的工具和实践。下面逐个说明。



上图来自于公众号文章：https://mp.weixin.qq.com/s/gs5ndvlMqM-Y4jg1_D2aFw 该文对Harness做了详细的讲解，本文不过多赘述；

这里我们只关注实践工具在其中的构成。

支柱一：上下文管理（Context Architecture）

> 问题：AI 的上下文窗口有限且贵，怎么让 AI 在对的时间看到对的信息？

实践	工具	说明
渐进式披露	`AGENTS.md`	写一个 ~100 行的目录文件，指向 `ARCHITECTURE.md`、`Rules` 等细分文档，别一次灌几千行

实践	工具	说明
结构化规范	Spec `.md` 文件（requirement.md / task.md）	把需求和设计决策写进 Git 仓库，变成 AI 随时能调取的"长期记忆"
变更隔离	`changes/` 目录	用 Proposals 机制把"增量变更"和"存量代码"隔开，减少对现有逻辑的误伤
知识分层	Skills 按需加载	技能信息分三层（描述 → 指令 → 详细步骤），按需逐步加载，省 Context
知识库挂载	知识库（iWiki 代码库 自定义文件）	把团队 Wiki、代码仓库、业务文档挂载为知识库，AI 对话时自动或手动引用，获取业务上下文
代码知识化	AI Wiki	基于代码库自动生成结构化知识文档，AI 不用逐文件阅读就能理解项目全貌

几个原则：

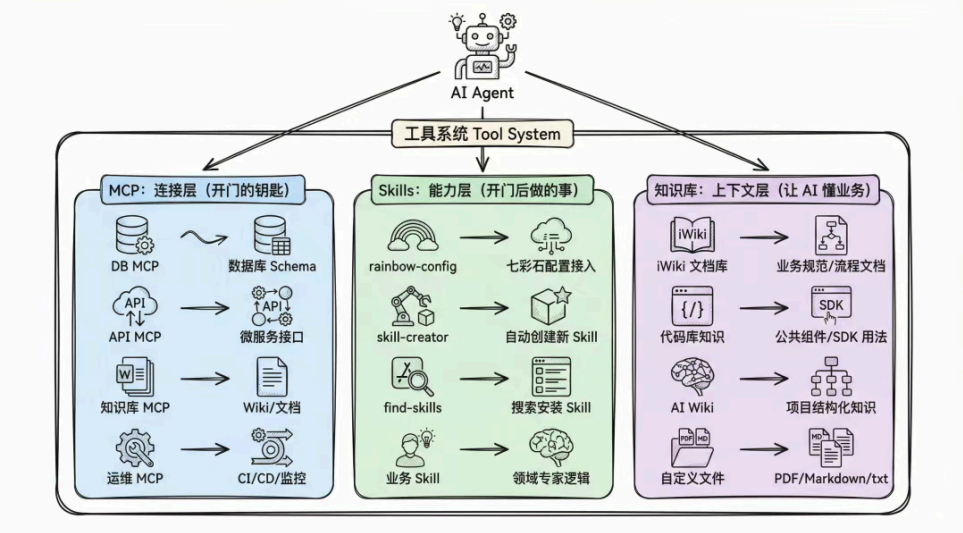
- 别给 AI 一个几千行的规范文件
- 建分层索引，让 AI 按需深入
- 把团队 Wiki 和业务文档挂载为知识库，让 AI 有业务上下文
- 把仓库知识当作"系统记录"（System of Record），别依赖聊天历史

OpenAI 自己踩过坑：早期试过"一个巨大的 AGENTS.md"，失败了。正确做法是拆成多个专注的文档，用目录索引串起来。

支柱二：工具系统（Tool System）

> 问题：AI 怎么触达代码仓库之外的真实世界，怎么具备特定领域的专业能力？

工具系统由三部分组成：MCP（连接外部世界）、Skills（封装专家经验）和知识库（注入业务上下文），三者配合构成 AI Agent 的完整能力体系。



MCP（Model Context Protocol）—— 连接外部数据源

MCP 类型	作用	典型场景
DB MCP	自动读取实时数据库 Schema	避免 AI 写出不存在的字段，生成准确的 SQL
Knowledge Base MCP	挂载团队内部文档	让 AI 有业务上下文，理解领域术语
API MCP	实时查询其他服务接口定义	微服务联调时，确保接口参数一致
运维 MCP	接入 CI/CD、监控系统	AI 可以直接触发构建、查看日志、分析告警

Skills（Agent Skills）—— 封装领域专家经验

Skills 是业务逻辑、领域知识和执行 SOP 的封装，让 AI 从"什么都会一点"变成"某个领域的专家"。

Skill 类型	作用	典型场景
工具接入类	封装内部工具链的接入规范	`rainbow-config`：按标准流程接入七彩石配置中心
代码生成类	固化特定模式的代码生成逻辑	按团队架构规范生成 CRUD 模块、中间件接入代码
元技能类	让 AI 能自我扩展	`skill-creator`：教 AI 根据现有代码创建新 Skill
搜索发现类	从社区发现可用能力	`find-skills`：从 80,000+ 技能库搜索并安装 Skill

知识库（Knowledge Base）—— 注入业务上下文

知识库是让 AI 从"通用模型"变成"懂业务的助手"的关键。挂载团队内部文档、代码仓库和业务资料后，AI 对话时能自动获取业务上下文，少猜多做。

知识库类型	数据来源	典型场景
iWiki 文档库	团队 Wiki 空间	挂载业务规范、技术方案、API 文档，AI 回答时自动引用
代码库知识	工蜂 Git 仓库	挂载公共组件（如 tRPC、七彩石 SDK），AI 生成代码时参考正确用法
AI Wiki	代码库自动生成	基于代码库自动生成结构化知识文档，快速理解项目架构和模块逻辑
自定义文件	Markdown PDF txt	上传需求文档、设计稿、会议纪要等，让 AI 有项目背景

知识库使用方式：

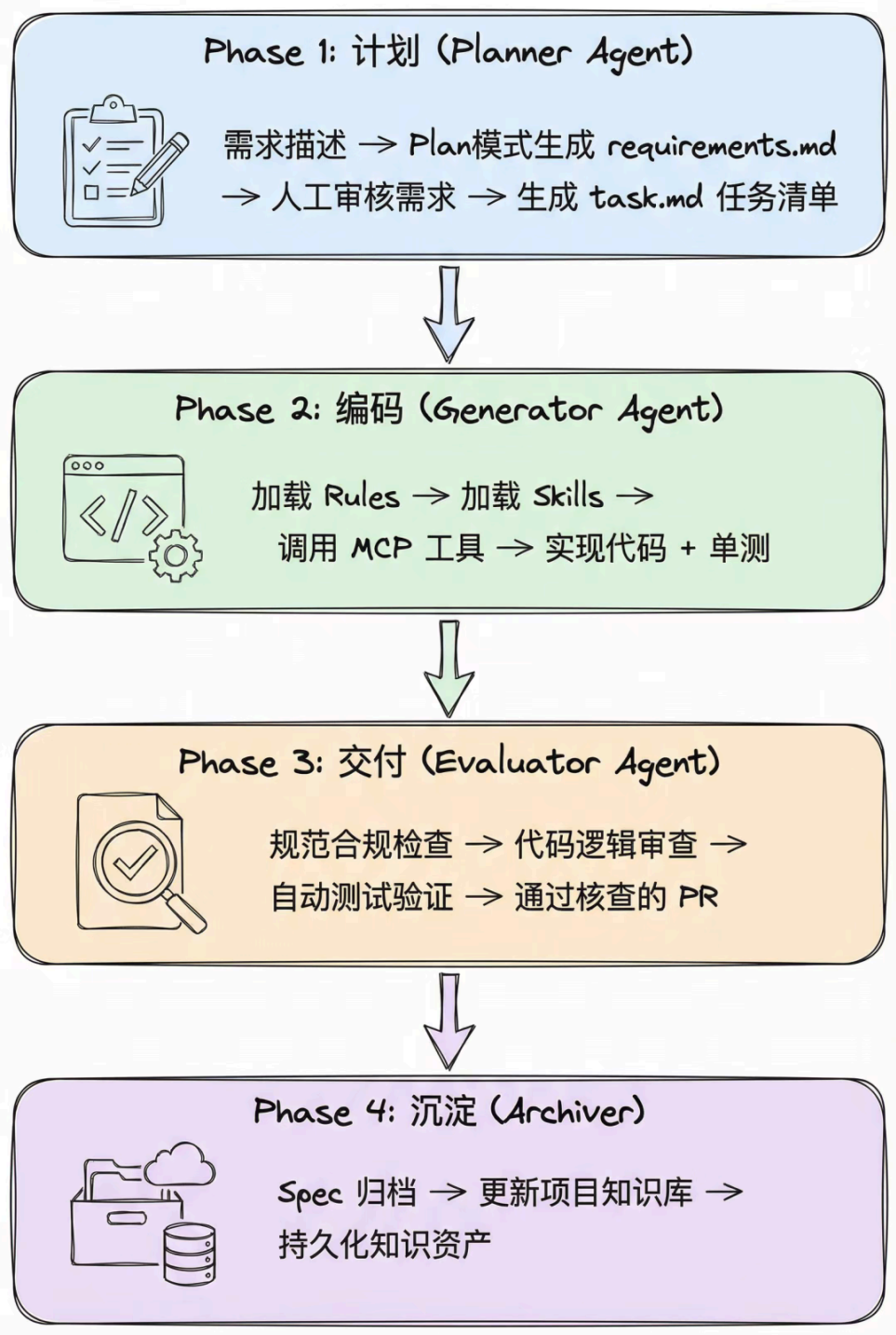
- **显式引用**：在对话中输入 `@KnowledgeBase` 选择特定知识库引用
- **自动引用**：开启自动参考开关，AI 对话时自动检索相关知识
- **团队共享**：通过 Knot 平台将知识库共享给团队/组织，统一业务认知

> **核心比喻**：MCP 是开门的钥匙，Skills 是开门后做的事情，知识库是进门前读的说明书。三者缺一不可——没有 MCP，AI 是闭门造车；没有 Skills，AI 有钥匙但不知道进门干什么；没有知识库，AI 进了门也不懂业务。

支柱三：执行编排与多 Agent 协作（Execution Orchestration）

> 问题：怎么让 AI 按部就班而不是乱写一气？怎么让多个 Agent 角色配合完成复杂任务？

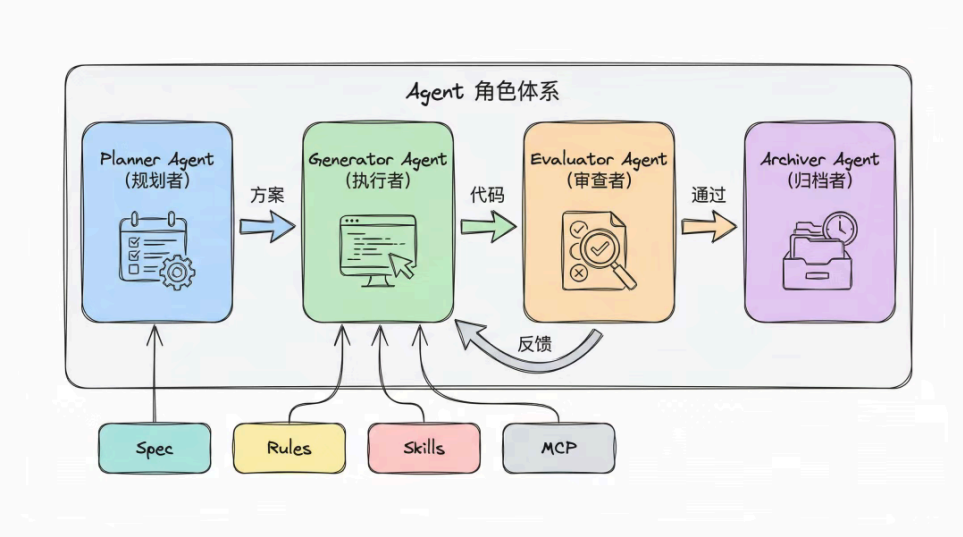
执行编排不只是选模式（Plan vs Agent），而是一套多 Agent 协作的标准化工作流。团队应该遵循“3+1 Phase”流程，每个阶段由不同角色的 Agent 负责：



"3+1 Phase" 标准化工作流：

阶段	输入	AI 操作	产出	协作模式
Phase 1: 计划	需求描述	Plan 模式生成 requirements.md，人工审核后创建 task.md	结构化方案文件	人类 Review 方案
Phase 2: 编码	任务清单	加载 Rules 和 Skills，调用 MCP 工具实现代码	源代码 + 单元测试	Generator Agent 执行
Phase 3: 交付	待合入代码	AI 自动做规范合规检查和代码逻辑审查	通过核查的 PR	Evaluator Agent 验收
Phase 4: 沉淀	已合并需求	自动把 Spec 归档，更新项目知识库	持久化知识资产	归档自动化

多 Agent 角色定义：



Agent 角色	职责	加载的 Harness
Planner	理解需求、拆解任务、生成方案	Plan 模式 + 项目 Spec
Generator	按方案写代码、写测试	Rules + Skills + MCP
Evaluator	代码审查、规范检查、测试验证	Rules + 验收标准
Archiver	归档变更、更新知识库	归档脚本 + Git

实际操作中：

- 用 **Plan 模式**做架构分析和大任务拆解（Planner 角色）
- 用 **Agent 模式**做具体功能的自动化实现（Generator 角色）
- 用 **AI Code Review** 做交付前的质量把关（Evaluator 角色）
- 遵循 **SDD 工作流**：`requirements.md → 人工审核 → task.md → 执行 → 归档`
- 每个任务必须有明确的"完成标准"（Acceptance Criteria）

支柱四：状态与记忆（State & Memory）

> 问题：怎么让 AI 在长周期开发中保持一致性？

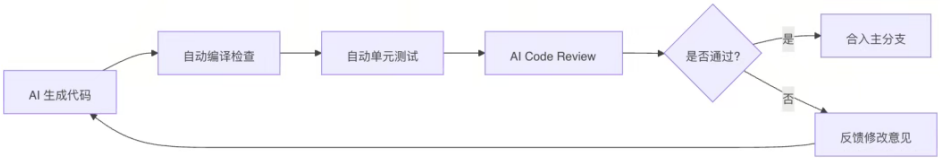
记忆类型	实现方式	生命周期
短期记忆	当前会话上下文	单次对话
中期记忆	Memories 功能	跨会话持久化
长期记忆	Git 仓库中的 Spec 文件	项目全生命周期
变更记忆	Spec Deltas（`changes/` 目录）	单次变更周期

实际操作中：

- 用 Git 记录规范变更（Spec Deltas），形成项目的长期记忆
- 用 Memories 功能让 AI 记住编程习惯和项目信息
- 每次变更归档后，自动更新 `.codebuddy/plan/` 下的归档记录

支柱五：评估与观测（Evaluation & Observability）

> 问题：怎么验证 AI 生成的代码是不是靠谱的？



评估分四层：

层次	检查内容	工具/方式
L1 语法	编译通过、Lint 检查	`go build` / `golangci-lint`
L2 逻辑	单元测试通过	`go test` / 自动生成测试用例
L3 规范	符合 Rules 约束	AI 自动合规检查
L4 架构	不破坏现有设计	人工 + AI 联合审查

实际操作中：

- 引入 AI 代码审查（CR），合入前自动检查规范合规性
- 代码写完后，自动编译和基础自测（闭环验证）
- 影响较大的改动，可以自动生成变更日志

支柱六：约束与恢复（Guardrails & Recovery）

> 问题：怎么防止 AI 越界操作，出错了怎么快速恢复？

约束分三级：

```
┌──────────────────────────────────────────┐
|  硬性红线（Rules - 不可违反）               |
|  "所有 API 必须包含 Swagger 注解"          |
|  "禁止在 Controller 层编写业务逻辑"        |
|  "所有数据库查询必须使用 Repository 模式"  |
├──────────────────────────────────────────┤
|  软性约束（Skills - 推荐遵循）              |
|  "优先使用项目已有的工具类"                |
|  "日志格式遵循团队统一标准"                |
├──────────────────────────────────────────┤
|  安全策略（Safety - 兜底保护）              |
|  "涉及数据库变更，优先生成 SQL 脚本"       |
|  "高风险操作前自动检测影响范围"            |
|  "重要操作自动备份"                        |
└──────────────────────────────────────────┘
```

恢复机制：

- 所有变更通过 Git 管理，随时可以回滚
- Spec Deltas 机制确保变更可追溯
- 编译失败时自动回退到上一个稳定状态

1.4 Harness 6 大支柱与工具链映射总表

支柱	核心问题	对应工具	团队实践
上下文管理	AI 看到什么信息？	Spec 文档 AGENTS.md 知识库	结构化规范 + 渐进式披露 + 业务知识挂载
工具系统	AI 能触达什么？	MCP Skills 知识库	DB/API 实时接入 + 知识库业务沉淀 + Skills 专家经验
执行编排与多 Agent 协作	AI 按什么顺序做？谁来做？	Plan 模式 SDD 工作流 多 Agent 角色体系	"3+1 Phase"：Planner → Generator → Evaluator → Archiver
状态与记忆	AI 记住什么？	Git + Memories + Spec Deltas	长期记忆持久化
评估与观测	AI 做得对不对？	自动测试 + AI CR	编译→测试→审查闭环
约束与恢复	AI 不能做什么？	Rules + Safety 策略	硬性红线 + 自动回滚

下文会详细讲解具体工具规范。

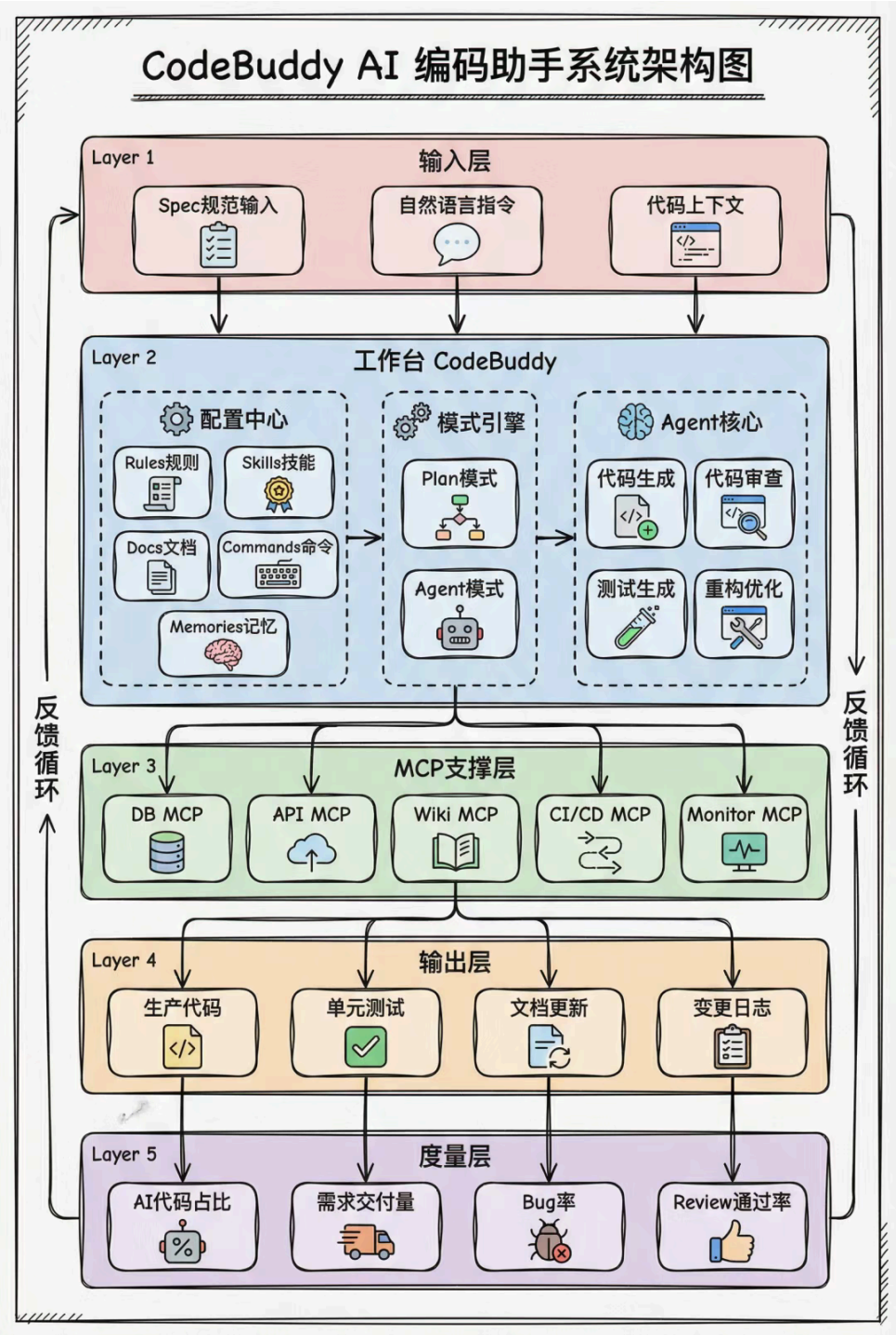
二、AI Coding 一体化架构

基于Harness Engineering 的 6 个支柱，这一章把它落地成一套完整的架构。这套架构定义了从"人的想法"到"能跑的代码"的全链路，算是团队 AI 辅助开发的技术蓝图。

说白了，AI 不是一个孤立的代码生成器，它是嵌在整个工程体系里的一个节点。架构的每一层都对应 Harness 的某个支柱，确保 AI 在约束下干活。

2.1 架构全景图

在开发实践过程中,我们整理了一个AI编码的整体架构图，从上到下分五层：输入层 → 工作台（CodeBuddy）→ 底层支撑（MCP）→ 输出层 → 度量层，数据自上而下流动，形成闭环：

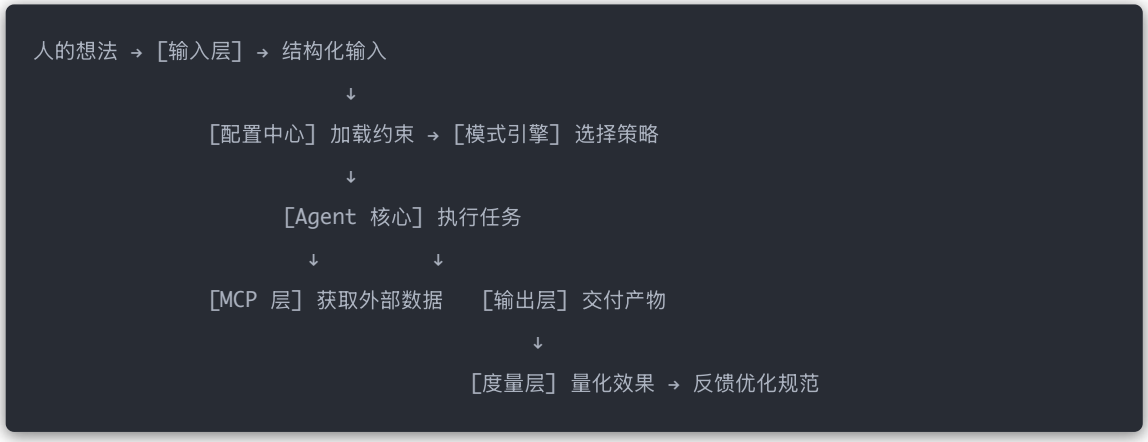


各层职责

以下表格说明架构中每一层的组件和职责：

层级	组件	职责
输入层	Spec 文档（requirement.md）/ 自然语言 / 代码上下文	把人的想法转成 AI 能理解的结构化输入
配置中心	Rules Skills Docs Commands Memories	加载 Harness 约束，让 AI 行为可控
模式引擎	Plan 模式 / Agent 模式	根据任务复杂度选执行策略
Agent 核心	代码生成 审查 测试 / 重构	执行具体的开发任务
MCP 层	DB API Wiki CI/CD Monitor	连接外部系统，突破代码仓库边界
输出层	代码 测试 文档 / 日志	交付可运行的工程产物
度量层	AI 代码占比 交付量 Bug 率	量化 AI 辅助开发的效果

数据怎么流转



注意，这不是单向流水线，而是一个闭环——度量层的数据会反馈到配置中心，推动 Rules 和 Skills 的迭代。比如度量发现 Bug 率上升了，团队就该检查是不是需要补新的 Rules 约束或者优化现有 Skills。

第二部分：落地实操

> 以上两章阐述了 Harness Engineering 的核心理念和 AI Coding 一体化架构的设计蓝图。理解了"为什么"和"是什么"之后，接下来最关键的问题就是"怎么做"。本部分聚焦于**如何一步步把规范落地到团队日常开发中**。每一节都包含具体的操作步骤、配置示例和验收标准，确保团队成员照着做就能跑通。

三、实施路线图（3 阶段渐进式）

落地不是一蹴而就的事。我们把整个过程拆成三个阶段，每个阶段有明确的目标和验收标准：

阶段	目标	周期	核心产出
第一阶段：基础建设	让团队每个人都能用上 AI Coding 工具，建立基本约束体系	1-2 周	CodeBuddy 安装 + team-harness 仓库 + 基础 Rules + 知识库配置
第二阶段：工具接入	接入 MCP、沉淀 Skills、实践 Plan 模式 SDD	2-4 周	MCP 接入 + Skills 沉淀 + Spec 驱动开发流程跑通

阶段	目标	周期	核心产出
第三阶段：持续优化	建立自演进的知识体系，实现知识飞轮效应	持续	度量看板 + 规范迭代机制 + 知识飞轮

四、第一阶段：基础建设（快速启动）

目标：让团队每个人都能用上 AI Coding 工具，并建立基本的约束体系。

4.1 CodeBuddy 安装与配置

4.1.1 IDE 插件安装

VSCode 安装：

1. 前往 CodeBuddy官网下载插件 `.vsix` 文件
2. 进入 VSCode → Extensions → `...` → Install from VSIX → 选择下载的插件
3. 按 `Command(⌘) + L` 或 `Ctrl + L`，底部出现 CodeBuddy 图标代表安装成功
4. 登录账号，确认 Plan 模式和 Agent 模式均可正常使用

JetBrains 系列 IDE 安装（GoLand PyCharm IDEA 等）：

1. 前往 CodeBuddy 官网下载 JetBrains 插件 `.zip` 文件（**注意：下载后不要解压**）
2. 进入 IDE → Plugins → ⚙ → Install Plugin from Disk → 选择下载的 `.zip` 文件
3. 底部出现 CodeBuddy 图标代表安装成功
4. 登录账号，确认对话功能正常

> ⚠️ Mac Safari 浏览器默认会自动解压 zip 文件，建议在 Safari 设置中取消勾选"下载后打开安全文件"。

4.1.2 CLI 工具安装（可选）

司内已集成三种顶级 CLI 编程工具，按需选择：

CLI 工具	安装命令	启动命令	配置目录
Claude Code Internal	`npm install -g --registry=https://xxx.com/	`claude-internal`	`~/.claude-internal/`
Gemini CLI Internal	`npm install -g --registry=https://xxx.com/`	`gemini-internal`	`~/.gemini/`
Codex CLI Internal	`npm install -g --registry=https://xxx.com/`	`codex-internal`	`~/.codex-internal/`

> 前置依赖：Node.js 20 或以上版本。三者都属于行业最强 AI Coding 工具，按个人习惯选择即可。

4.1.3 CodeBuddy 核心配置

安装完成后，需要进行以下核心配置，让 CodeBuddy 发挥最大效能：

1. 模型选择

在对话框左下角切换模型。推荐策略：

场景	推荐模型	说明
复杂编码任务	Claude-4.6-Sonnet/Opus(更强) / GPT-5.4	外部模型，编程能力一流，但会外传代码上下文
简单问题 / 敏感业务	DeepSeek-V3.2 GLM-4.7 HY-2.0	内部部署，代码不出域，安全有保障
不确定选哪个	Auto（智能自动选择）	基于问题复杂度自动匹配最优模型

> ⚠️ 安全提醒：Claude、GPT、Gemini 等外部模型会发送代码上下文到外部，敏感业务请使用内部部署模型。

2. Memories 配置（记忆功能）

Memories 让 CodeBuddy 记住你的编码习惯和项目信息，跨会话持久化。

开启方式：

1. 在 CodeBuddy 设置页面，选择 Memories 选项
2. 确认 Memories 开关已开启

主动记忆： 在 Agent 模式对话中，直接告诉 CodeBuddy 需要记住的信息：

```
请记住：
1. 我习惯使用 Go 语言开发，项目使用 gin 框架
2. 代码注释使用中文
3. 变量命名使用 camelCase 风格
4. 所有 API 返回统一使用 pkg/response 包的标准格式
```

管理记忆： 在 CodeBuddy 设置页面 → Memories，可以查看、编辑、删除已保存的记忆。

3. Commands 配置（指令式交互）

Commands 是将高频开发任务封装为可复用命令的能力，本质是"可被快速触发的标准化 Prompt"。

创建 Command：

1. 在对话框输入 `/`，选择"新增 Command"
2. 输入 Command 名称（建议使用英文命名）

3. 填入 Command 内容（即预设的 Prompt）

推荐的团队 Commands：

Command 名称	用途	触发方式
`/init`	为项目初始化 AI 使用手册（自动生成 Rules）	新项目首次使用时
`/pre-mr-checklist`	代码提交前安全 & 漏洞检测	提交 PR 前
`/spec-create`	创建需求 Spec 文档	新需求开发时
`/spec-plan`	基于 Spec 生成任务清单	需求审核通过后

`/init` Command 示例内容：

```
请分析此代码库，并在当前代码库 `.codebuddy/rules` 目录下创建 global.md 文件，
该文件将提供给未来的 CodeBuddy 实例在此代码库中运行使用。

需要补充的内容：
1. 将经常使用的命令包括在内，例如如何构建、如何进行代码检查以及如何运行测试
2. High-level 的代码架构和结构，重点在于"宏观"的架构设计

使用说明：
- 如果该文件已经存在，请对其进行改进
- 不要包含通用的开发实践
- 确保该文件前有以下头部元数据：
---
# CodeBuddy Rules
type: always
---
```

> ⚠️ 验收标准：每位团队成员能在 IDE 中正常唤起 CodeBuddy 对话框，Token 使用量正常，能展示一个简单项目实现的 prompt。

4.2 创建 team-harness 仓库

这是团队规范的唯一真实来源，所有 Rules、Skills 模板、AGENTS.md 模板都集中管理在这里。

Step 1：初始化仓库结构

```
# 创建仓库
mkdir team-harness && cd team-harness
git init

# 创建标准目录结构
mkdir -p rules/{global,golang,python,frontend}
mkdir -p skills/{common,business}
mkdir -p templates
```

```
mkdir -p docs

# 创建核心文件
touch rules/global/base.md
touch rules/golang/go-backend.md
touch templates/AGENTS.md
touch templates/project.md
touch README.md
```

最终目录结构：

```
team-harness/
├── rules/                      # 团队 Rules 集合
|   ├── global/                 # 全局通用规则
|   |   └── base.md             # 基础规范（所有项目必须加载）
|   ├── golang/                 # Go 语言专用规则
|   |   └── go-backend.md
|   ├── python/                 # Python 专用规则
|   └── frontend/               # 前端专用规则
├── skills/                     # 团队 Skills 集合
|   ├── common/                 # 通用 Skills
|   |   ├── skill-creator/      # Skill 创建器
|   |   └── find-skills/        # Skill 搜索器
|   └── business/               # 业务 Skills
|       └── rainbow-config/     # 七彩石配置接入
├── templates/                  # 模板文件
|   ├── AGENTS.md               # AI 说明书模板
|   └── project.md              # 项目描述模板
├── docs/                       # 使用文档
|   └── onboarding.md           # 新人上手指南
└── README.md
```

Step 2：编写同步脚本

在业务项目中通过脚本自动拉取最新规范：

```
#!/bin/bash
# sync-harness.sh - 同步团队规范到当前项目
HARNESS_REPO="git@xxx.com"
HARNESS_DIR=".harness-upstream"

# 拉取最新规范
if [ -d "$HARNESS_DIR" ]; then
    cd $HARNESS_DIR && git pull && cd ..
else
    git clone $HARNESS_REPO $HARNESS_DIR
fi
```

```
# 同步 Rules 到项目
mkdir -p .codebuddy/rules
cp $HARNESS_DIR/rules/global/*.md .codebuddy/rules/
cp $HARNESS_DIR/rules/golang/*.md .codebuddy/rules/  # 按语言选择

# 同步 Skills 到项目
mkdir -p .codebuddy/skills
cp -r $HARNESS_DIR/skills/common/* .codebuddy/skills/

echo "✅ 团队规范同步完成"
```

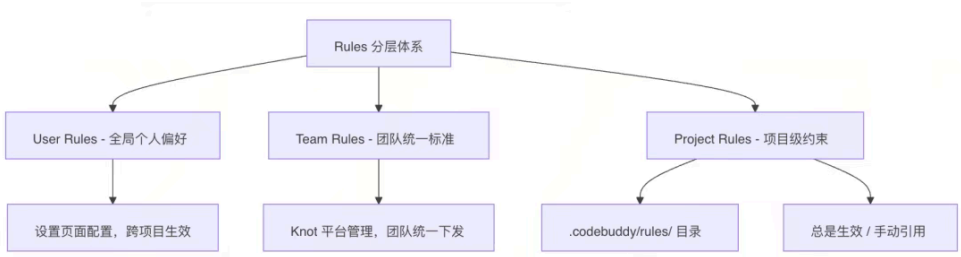
Step 3：配置 CI 自动同步（可选）

在项目的 CI 流水线中加入自动同步步骤，确保每次构建前规范都是最新的。

4.3 Rules 配置（全局与项目级约束）

Rules 是 AI 在每次交互中必须加载的全局约束，相当于 AI 必须遵守的"法律"。CodeBuddy 支持三个层级的 Rules：

4.3.1 Rules 分层体系



层级	作用域	配置方式	加载方式
User Rules	所有项目（个人）	CodeBuddy 设置页面 → Rules	每次对话自动带入
Team Rules	团队所有成员	Knot 平台管理下发	按 type 配置（always / manual）
Project Rules	单个项目	`.codebuddy/rules/` 目录下的 `.md` 文件	总是生效 或 手动 @引用

4.3.2 User Rules 配置

1. 点击 CodeBuddy 对话面板的设置齿轮图标
2. 进入 Rules 设置页面
3. 添加个人偏好规则，也可使用平台预置的 Rules 快速生成后微调

```
# 个人偏好示例
1. 回复使用中文
2. 代码注释使用中文
```

```
3. 优先使用 Go 标准库
4. 变量命名使用 camelCase
```

4.3.3 Team Rules 配置（通过 Knot 平台）

Team Rules 由团队管理员在 Knot 平台统一管理和下发，确保团队所有成员遵循一致的标准。

配置步骤：

1. 前往 Knot Rules 管理页面
2. 点击「新建 Team Rule」
3. 填入 Rule 内容，头部必须包含 Rule Type Header：

```
---
type: always
---
# 团队 Go 后端开发规范

## 架构约束
1. 严格遵循分层架构：Controller → Service → Repository → Model
2. 禁止在 Controller 层编写业务逻辑
...
```

1. 提交审批，审批通过后 Team Rule 自动生效
2. 团队成员的 CodeBuddy 会自动加载已生效的 Team Rules

> 💡 Team Rule 的 `type` 支持 `always`（总是生效）和 `manual`（手动引用）两种模式。

4.3.4 Project Rules 配置

创建方式：

1. 在 CodeBuddy 对话面板中点击「新增 Project Rule」
2. 输入 Rule 内容（注意不要修改头部元数据）
3. 设置生效范围：
   - **总是生效**：每次对话自动带入
   - **手动指定**：需要在对话时 `@Rules` 选择

Rules 文件结构规范：

```
---
description: "Go 后端开发通用规范"
globs: "**/*.go"
alwaysApply: true
---
```

```
# Go 后端开发规范

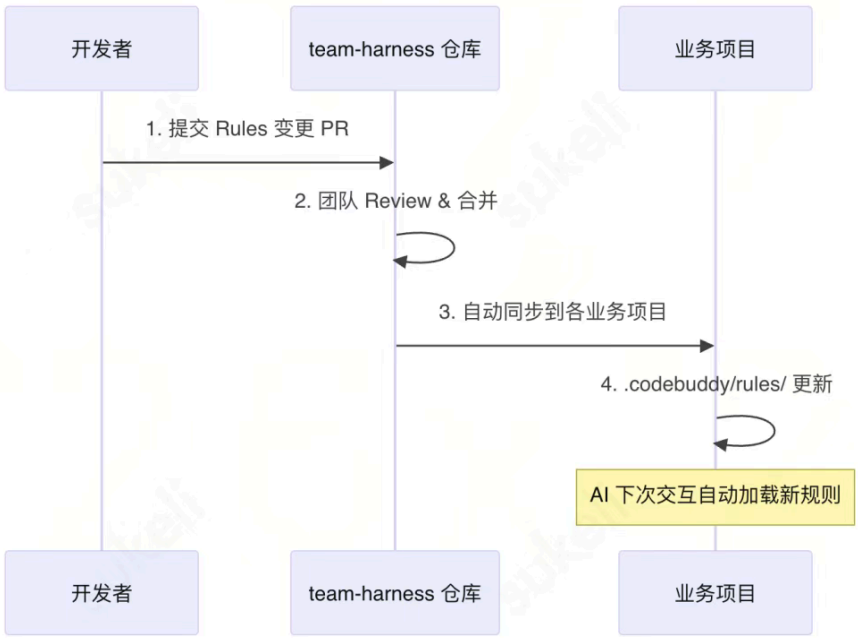
## 一、架构约束（硬性红线）
1. 严格遵循分层架构：Controller → Service → Repository → Model
2. 禁止在 Controller 层编写业务逻辑，Controller 只负责参数校验和响应封装
3. 所有数据库操作必须通过 Repository 层，禁止在 Service 中直接写 SQL
4. 所有对外 API 必须包含 Swagger 注解

## 二、代码风格
1. 函数/方法必须有简要注释说明用途
2. 错误处理不允许使用 _ 忽略，必须显式处理或向上传递
3. 变量命名使用 camelCase，常量使用 ALL_CAPS
4. 单个函数不超过 80 行，超过则拆分

## 三、安全策略
1. 涉及数据库变更时，优先生成 SQL 变更脚本，而非直接执行
2. 删除、移动文件等操作无需额外确认，但涉及数据库结构修改必须确认
3. 所有敏感配置（密钥、连接串）必须通过配置中心读取，禁止硬编码

## 四、开发行为
1. 添加新功能前，必须先分析现有代码库，优先复用已有模块
2. 代码变更范围最小化，一次 PR 只解决一个问题
3. 每次变更必须附带清晰的 commit 信息
4. 新增功能必须同步编写单元测试
```

4.3.5 Rules 的保存与复用流程



业务项目team-harness 仓库开发者业务项目team-harness 仓库开发者AI 下次交互自动加载新规则1. 提交 Rules 变更 PR2. 团队 Review & 合并3. 自动同步到各业务项目4. .codebuddy/rules/ 更新

验证 Rules 生效：

```
# 在 CodeBuddy 中测试
你好，请告诉我当前加载了哪些 Rules?
```

AI 应能识别并列出已加载的规则文件。

4.4 编写 AGENTS.md

`AGENTS.md` 是 AI 的"说明书"，控制在 ~100 行以内，当目录索引用，指向更细分的文档。

创建文件 `AGENTS.md`（放在项目根目录）：

```
# AI 开发助手说明书

## 项目概述
本项目是 [项目名称]，基于 Go 微服务架构，使用 [框架名] 框架。

## 架构说明
- 分层架构：Controller → Service → Repository → Model
- 详细架构文档：参见 `docs/ARCHITECTURE.md`

## 目录结构
- `internal/` - 业务逻辑（按服务拆分子目录）
- `pkg/` - 公共工具库
- `api/` - API 定义（Proto/Swagger）
- `configs/` - 配置文件
- `scripts/` - 脚本工具

## 开发规范
- 代码规范：参见 `.codebuddy/rules/go-backend.md`
- 数据库规范：所有查询走 Repository 层
- 错误处理：统一使用 `pkg/errors` 包装错误

## 常用命令
- 编译：`go build ./...`
- 测试：`go test ./...`
- Lint：`golangci-lint run`

## 当前进行中的需求
- 参见 `.codebuddy/plan/` 目录下的活跃需求

## 注意事项
- 添加新功能前，先检查 `pkg/` 下是否已有可复用的工具
- 数据库变更必须先生成 SQL 脚本
- 所有 API 变更需要更新 Swagger 文档
```

> ⚠️ AGENTS.md 是目录索引，不是百科全书。保持精简，让 AI 按需深入查阅具体文档。

4.5 知识库配置（详细实操）

知识库是让 AI 有业务上下文的核心手段。挂载团队内部文档、代码库和业务知识后，AI 能从"通用智能"变成"懂你业务的专家"。

4.5.1 知识库类型与适用场景

知识库类型	数据来源	适用场景	配置入口
iWiki 文档库	团队 Wiki 空间	业务文档、技术方案、API 说明、运维手册	Knot 平台
工蜂代码库	Git 仓库代码	公共组件 SDK、框架源码、参考实现	Knot 平台
自定义文件	Markdown txt PDF	需求文档、设计稿、会议纪要、领域知识	Knot 平台
AI Wiki	基于代码库自动生成	项目架构理解、模块逻辑梳理、新人上手	CodeBuddy 内置

4.5.2 在 Knot 平台创建团队共享知识库

Step 1：创建知识库

1. 前往 Knot 知识库管理页面
2. 点击「添加知识库」
3. 选择知识库类型（iWiki 工蜂代码库 自定义文件）
4. 填入知识库信息：
    - **iWiki 类型**：填入 iWiki 空间地址
    - **工蜂代码库类型**：填入 Git 仓库地址和分支
    - **自定义文件类型**：上传 Markdown txt PDF 文件

Step 2：配置共享范围

1. 在知识库详情页，开启「共享开关」
2. 选择需要分享的组织/团队
3. 提交后等待管理员审批，审批通过即完成团队共享

Step 3：配置数据源

在知识库的「数据源配置」页面，可以配置多种数据源：

- **需求**：支持 TAPD 项目
- **代码**：支持工蜂 Git 仓库（填入仓库地址和分支）
- **文档**：支持 iWiki 空间
- **可观测**：支持智研项目

4.5.3 在 CodeBuddy 中启用知识库

Step 1：进入知识库设置

在 CodeBuddy 对话面板中，点击设置图标 → 进入「知识库」选项。

Step 2：开启知识库

1. 在知识库列表中，开启需要的公共知识库和个人知识库
2. 配置自动引用开关（推荐开启，AI 会自动参考相关知识）

Step 3：使用知识库的两种方式

```
# 方式一：显式引用（精确控制）
# 在对话输入框中输入 @KnowledgeBase，选择特定知识库
@团队技术文档 请帮我分析当前项目的缓存策略是否合理

# 方式二：自动引用（省心省力）
# 开启自动参考开关后，AI 会根据问题自动检索相关知识
请帮我实现用户鉴权模块，参考团队现有的鉴权方案
```

4.5.4 开通 AI Wiki（推荐）

AI Wiki 是基于代码库自动生成的结构化知识库，帮助团队成员快速理解项目架构：

1. 在 CodeBuddy 右上角菜单中打开 AI Wiki
2. 按指引为当前代码库开通 AI Wiki（索引通常在 24h 内完成）
3. 开通后可直接在 IDE 中浏览项目文档，点击文件跳转到源码
4. 通过 `@AIWiki` 向 AI Wiki 提问，快速了解项目模块逻辑

4.5.5 推荐的团队知识库清单

优先级	知识库名称	类型	内容
P0	团队技术文档	iWiki	架构设计、技术方案、接口文档
P0	核心公共库	工蜂代码库	tRPC SDK、七彩石 SDK、北极星 SDK 等
P1	业务需求文档	自定义文件	产品需求文档、设计稿
P1	项目 AI Wiki	AI Wiki	基于代码库自动生成的结构化文档
P2	运维手册	iWiki	部署流程、监控告警、故障处理

> ⚠️ 验收标准：团队成员在 CodeBuddy 中提问业务相关问题时，AI 能自动引用知识库内容给出准确回答，而不是泛泛而谈。

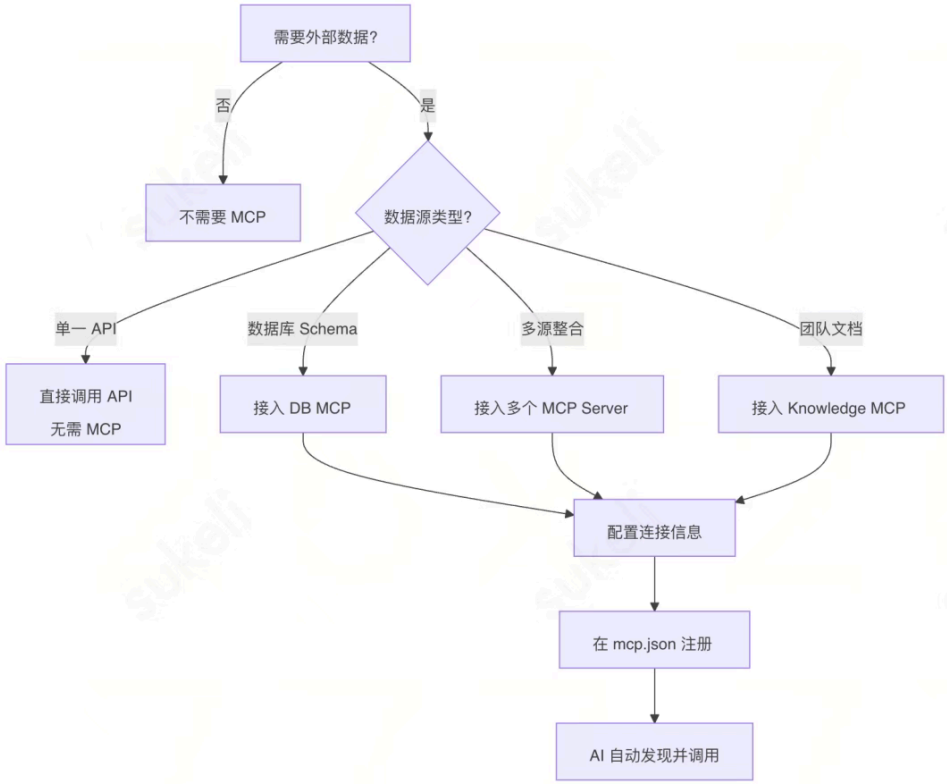
五、第二阶段：工具接入（深度集成）

目标：接入 MCP、沉淀 Skills、初始化 Spec 目录结构，在试点项目中实践 Plan 模式 SDD。

5.1 MCP 配置（上下文边界突破）

MCP（Model Context Protocol）是 AI 的"感知触手"，让 AI 能触达代码仓库之外的真实世界。

5.1.1 MCP 接入决策



⚠️ **何时不该用 MCP**：

- 只写一个简单脚本查天气 → 直接调 API
- 纯逻辑推理、创意写作、代码生成 → MCP 几乎没有用武之地
- 引入 MCP 的复杂度 > 它解决的问题 → 不用

5.1.2 CodeBuddy 插件端 MCP 配置

Step 1：打开 MCP 配置

1. 在 CodeBuddy 对话面板点击「对话设置」
2. 点击「添加 MCP」
3. 编辑 `mcp.json` 配置文件

Step 2：配置 mcp.json

MCP 支持三种协议类型：

stdio 类型（本地命令行工具）：

```
{
  "mcpServers": {
```

```
    "db-mysql": {
      "command": "npx",
      "args": ["-y", "@anthropic/mcp-server-mysql"],
      "env": {
        "MYSQL_HOST": "127.0.0.1",
        "MYSQL_PORT": "3306",
        "MYSQL_USER": "readonly_user",
        "MYSQL_PASSWORD": "${DB_PASSWORD}",
        "MYSQL_DATABASE": "your_database"
      },
      "timeout": 10000,
      "transportType": "stdio"
    }
  }
}
```

streamable-http 类型（推荐，远程服务）：

```
{
  "mcpServers": {
    "gump-tool": {
      "url": "http://127.0.0.1:3000/mcp",
      "timeout": 10000,
      "headers": {
        "Authorization": "Bearer your-token"
      },
      "transportType": "streamable-http"
    }
  }
}
```

sse 类型（逐步废弃，优先使用 streamable-http）：

```
{
  "mcpServers": {
    "legacy-server": {
      "url": "http://0.0.0.0:3001/sse",
      "headers": {},
      "timeout": 10000,
      "transportType": "sse"
    }
  }
}
```

> ⚠️ 注意事项：

> - `timeout` 单位是 ms，默认 10s，最大 300s
> - stdio 类型的 `args` 必须拆开，不能合并为一个字符串
> - MCP 只在 Agent 模式下生效，提问时需要打开 Agent

Step 3：司内常用 MCP 配置参考

```
{
  "mcpServers": {
    "gongfeng": {
      "command": "npx",
      "args": ["-y", "@tencent/tgit-mcp-server@latest"],
      "env": {
        "GONGFENG_ACCESS_TOKEN": "你的工蜂密钥"
      }
    },
    "iWiki": {
      "headers": {
        "Authorization": "Bearer 你的太湖 token"
      },
      "type": "http",
      "url": "https://prod.xxx.com"
    },
    "tapd": {
      "headers": {
        "X-Tapd-Access-Token": "TAPD 个人Token",
        "X-Keep-Links": "true"
      },
      "type": "http",
      "url": "http://mcp.xxx.com"
    }
  }
}
```

> 更多 MCP 可前往 Knot MCP 市场 获取。

5.1.3 CLI 端 MCP 配置

Claude Code Internal：

- 用户级配置：`~/.claude-internal/.claude.json`
- 项目级配置：项目根目录下的 `.mcp.json`

Gemini CLI Internal：

- 配置文件：`~/.gemini/settings.json`

- 注意：CLI 配置 Streamable Http 格式的 MCP，`url` 需要写作 `httpUrl`

5.1.4 验证 MCP 连接

在 CodeBuddy 中测试：

```
请通过 DB MCP 读取当前数据库中 users 表的结构，列出所有字段名和类型。
```

如果失败，检查：

- MCP Server 是否正常启动（查看 CodeBuddy 输出面板的日志）
- 数据库连接信息是否正确
- 网络是否可达
- 是否已开启 Agent 模式

5.1.5 团队 MCP 接入清单

优先级	MCP Server	接入目的	验收标准
P0	DB MCP	AI 实时读取数据库 Schema	AI 能准确描述任意表结构
P0	工蜂 MCP	读取代码仓库、Issue、MR	AI 能读取 Issue 并给出实现思路
P1	iWiki MCP	挂载团队 Wiki 文档	AI 能回答业务领域问题
P1	TAPD MCP	读取需求和任务	AI 能读取需求并生成 Spec
P2	CI/CD MCP	触发构建和查看日志	AI 能执行构建并分析失败原因

5.2 Knot 平台配置（智能体与知识管理中枢）

Knot 平台是 CodeBuddy 生态的管理中枢，提供知识库、MCP、Rules、Skills、智能体等核心能力的统一管理。

5.2.1 Knot 平台核心功能一览

功能模块	入口	作用
智能体	knot.xxx.com	创建、管理、共享自定义智能体
研效知识库	knot.xxx.com	创建和管理团队知识库
MCP 市场	knot.xxx.com	发现和安装 MCP Server
Rules 市场	knot.xxx.com	获取和管理 Rules
Skills	knot.xxx.com	管理 Agent Skills

5.2.2 在 Knot 创建自主规划式智能体

自主规划式智能体能自主分析任务并制定执行计划，适合复杂多变的场景。

Step 1：新建智能体

1. 前往 Knot 智能体页面
2. 点击「+ 新建智能体」

3. 选择「自主规划」类型

Step 2：配置智能体

在智能体配置页面，填写以下信息：

配置项	说明	示例
智能体名称	简洁明了的名称	"团队需求评审智能体"
智能体描述	准确描述职责和能力（影响子智能体匹配）	"基于 TAPD 需求与代码库，评审需求完整性和可行性"
Prompt	详细的角色设定和行为指引	包含身份、目标、职责范围、操作指导
知识库	选择关联的知识库	团队技术文档、项目知识库
MCP 服务	选择需要的 MCP 工具	TAPD MCP、工蜂 MCP
Rules	选择适用的 Rules	团队编码规范
Skills	选择需要的 Skills	skill-creator 等
Client 工具	选择客户端工具	读取文件、执行命令等

Step 3：发布智能体

配置完成后，点击右上角「发布更新」。

5.2.3 配置智能体使用渠道

Knot 智能体支持多种使用渠道：

使用渠道	适用场景	配置方式
网页对话	日常使用、调试	默认可用，无需额外配置
企微智能机器人	团队群聊、私聊	配置 Bot ID 和 Secret
API 调用	集成到现有系统	获取 API 接口和密钥
网页 URL	分享给外部用户	生成独立网页链接
Knot CLI	命令行使用	安装 Knot CLI 工具
流水线	CI/CD 集成	在蓝盾/QCI 流水线中配置
定时运行	自动化任务	设置定时任务频率

企微智能机器人配置步骤：

1. 进入企微工作台 → 搜索「智能机器人」→ 创建机器人
2. 选择「手动创建 - API 模式」
3. 设置机器人基础信息
4. 将 Bot ID 和 Secret 填写到 Knot 智能体的「使用配置」中
5. **先保存智能机器人配置，再保存 Knot 配置**

6. 等待 5-8s 显示「已连接」后即可使用

5.2.4 智能体团队共享

1. 在智能体详情页 → 使用配置 → 权限配置
2. 编辑「可使用」权限，添加团队成员
3. 工作区也支持共享，在工作区管理页面开启共享开关

5.3 CodeBuddy 子智能体（SubAgent）配置

子智能体是 CodeBuddy 的核心协作能力——让多个专业智能体在对话中自动配合完成复杂任务。

5.3.1 什么是子智能体

在日常开发中，我们经常遇到固定的开发场景（需求分析、架构规划、国际化改造、重构等），反复处理这些任务时需要反复编写相同提示词、引用知识库并选择工具。

子智能体解决这个问题：根据开发场景，灵活组合提示词、工具和知识库打造业务专属智能体，启用后可在默认 Agent 对话时**根据对话任务动态调用合适的子智能体**协作完成任务。

5.3.2 创建自定义智能体

Step 1：创建智能体

1. 在 CodeBuddy 对话框左下角模式选择，点击「创建智能体」
2. 也可在对话面板顶部设置 → 对话 → 拉到底部的「自定义智能体」

Step 2：配置智能体

填写智能体的基本信息，组合可调用的工具、MCP、知识库：

配置项	说明	注意事项
名称	智能体名称	简洁明了
描述	职责描述	**非常重要**，会依据描述来匹配智能体
Prompt	行为指引	定义角色、能力、约束
工具	可调用的工具	按需选择
MCP	可调用的 MCP 服务	按需选择
知识库	关联的知识库	尽量选择与场景关联度高的，少而精

> ⚠️ 知识库选择后会在此智能体对话时主动引用，尽量选择仅与此场景关联度较高的知识库。未选择的知识库也可在对话时主动 @引用。

5.3.3 启用为子智能体（SubAgent）

如果需要多个智能体配合完成更复杂的工作场景，可以启用子智能体自动调用：

1. 给智能体添加**准确的职责描述**（此描述很重要，会依据描述来匹配智能体）
2. 勾选「子智能体」选项

提升子智能体被调用概率的技巧：

在描述中增加触发条件，例如：

```
当用户提出与数据库 / 数据查询 / 报表 / EDA 相关的请求时，必须调用我
```

5.3.4 推荐的团队子智能体配置

子智能体名称	职责描述	关联知识库	关联 MCP
需求分析专家	分析需求文档，生成 requirements.md	业务需求文档	TAPD MCP
架构设计专家	分析项目架构，给出设计建议	团队技术文档、AI Wiki	-
数据库专家	数据库设计、SQL 优化、Schema 分析	-	DB MCP
Code Review 专家	代码审查，检查规范合规性	团队编码规范	工蜂 MCP
运维排障专家	分析日志、定位问题、给出修复建议	运维手册	监控 MCP

5.3.5 公开分享智能体

创建的智能体可以通过 Knot 平台分享给团队：

1. 访问 Knot 智能体管理页面
2. 选择要分享的智能体（来自 CodeBuddy 创建的会有"CodeBuddy 智能体"标识）
3. 进入使用配置，编辑可见（可使用）范围
4. 团队成员在 Knot 平台收藏后，智能体会出现在 CodeBuddy 自定义智能体列表中

5.4 Skills 配置

Skills 是给 AI 的操作手册——把团队的专家经验、最佳实践和操作流程固化成 AI 可执行的指令。

5.4.1 Skill 文件结构规范

```
---
name: "rainbow-config"
description: "七彩石（Rainbow）配置中心的连接、查询和更新操作。
当需要对七彩石配置进行以下操作时使用：
(1) 初始化/连接配置中心
(2) 查询分组配置（KV 型或 Table 型）
(3) 获取/设置单个配置参数
(4) 添加配置变更监听"
---

# 七彩石配置接入 Skill

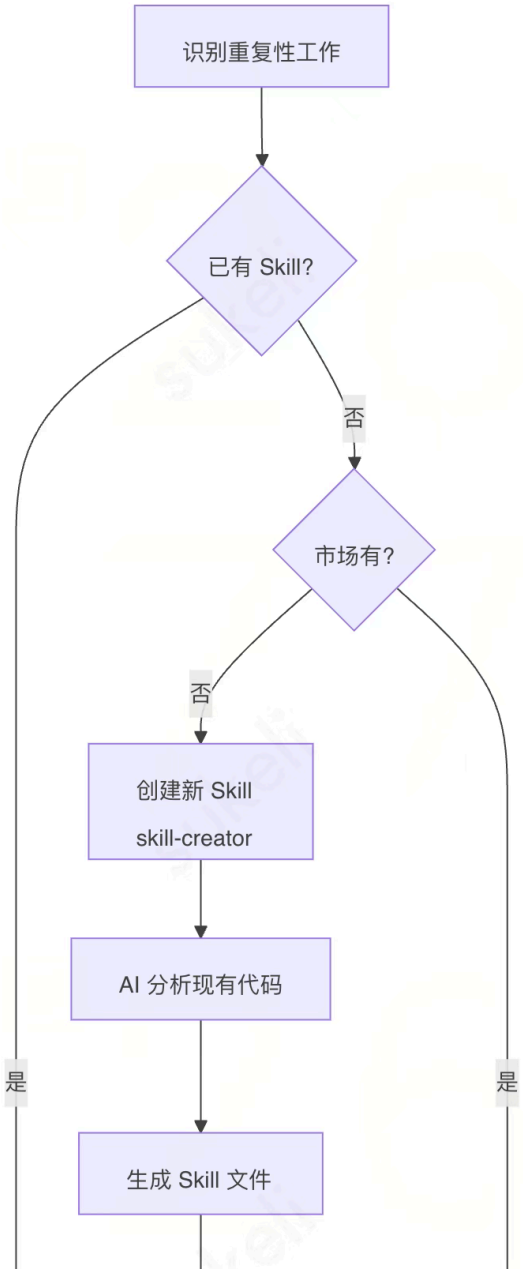
## 前置条件
```

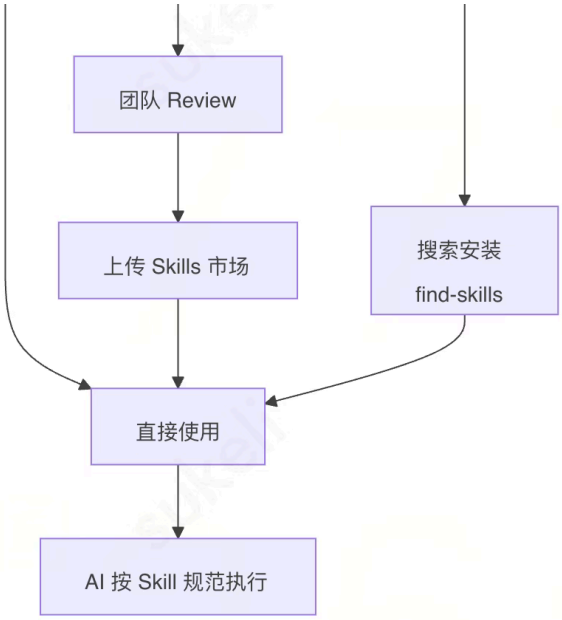
```
- 项目已引入 `pkg/rainbow` 包
- 已配置七彩石 AppID 和 Group

## 操作步骤
### Step 1: 初始化连接
[具体代码模板和说明...]
### Step 2: 查询配置
[具体代码模板和说明...]
### Step 3: 监听变更
[具体代码模板和说明...]

## 注意事项
- 配置缓存策略
- 错误处理规范
- 降级方案
```

5.4.2 Skills 创建与复用流程





5.4.3 Skill 创建实操

Step 1：安装 skill-creator

在 CodeBuddy 的 Skills 管理界面中搜索并安装 `skill-creator`。

Step 2：让 AI 分析现有代码并创建 Skill

```
我需要针对 pkg/rainbow 这个七彩石配置工具包创建一个 Skill。
请分析这个包的代码，按照 skill-creator 的规范生成一个标准的 Skill 文件。
```

Step 3：审查生成的 Skill 文件

检查 AI 生成的 Skill 是否包含：

- ✅ 准确的 `name` 和 `description`（决定 AI 何时触发此 Skill）
- ✅ 完整的前置条件说明
- ✅ 分步骤的操作指引
- ✅ 代码模板和配置示例
- ✅ 注意事项和错误处理

Step 4：验证 Skill 效果

```
我需要在当前项目中接入七彩石配置中心，读取 app_config 分组下的所有配置。
```

AI 应自动识别并加载 `rainbow-config` Skill，按照规范生成接入代码。

Step 5：上传到团队 Skills 仓库

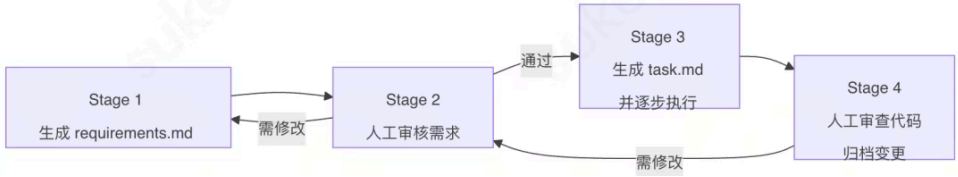
```
cp -r .codebuddy/skills/rainbow-config/ /path/to/team-harness/skills/business/
cd /path/to/team-harness
```

```
git add skills/business/rainbow-config/
git commit -m "feat: 新增七彩石配置接入 Skill"
git push
```

5.5 Spec 与 Plan 模式（规范驱动开发）

Plan 模式是实现规范驱动开发的核心手段：在 AI 动手写代码之前，先生成结构化的需求文档和任务清单，经过人工审核确认后，再按计划逐步执行。

5.5.1 Plan 模式开发流程（4 Stage）



阶段	操作	模式	产出
Stage 1	描述需求，AI 生成需求文档	Plan 模式	`.codebuddy/plan/feat-xxx/requirements.md`
Stage 2	人工逐项审核需求文档	人工审查	审核通过的 requirements.md
Stage 3	AI 生成任务清单并逐步执行	Agent 模式	`.codebuddy/plan/feat-xxx/task.md` + 源代码
Stage 4	人工审查代码，归档变更	人工审查	归档文档 + 变更日志

5.5.2 实操演练：以"新增用户操作日志模块"为例

Step 1：切换到 Plan 模式，描述需求

```
请使用 Plan 模式，分析以下需求并生成 .codebuddy/plan/feat-operation-log/requirements.md：

新增用户操作日志模块，要求：
1. 记录用户的关键操作（登录、修改资料、删除数据等）
2. 支持按用户ID、操作类型、时间范围查询日志
3. 提供管理后台的日志列表 API（分页）
4. 日志数据保留 90 天，过期自动清理

请明确：功能边界、API 接口定义、数据库表结构、异常处理策略和验收标准。
```

Step 2：审核 AI 生成的 requirements.md

```
□ 需求理解是否准确？有没有多做或少做？
□ API 接口路径是否符合团队 RESTful 规范？
□ 数据库表结构字段命名是否符合团队规范？索引设计是否合理？
□ 90 天自动清理的实现方案是否可行？
```

```
□ 异常处理是否覆盖了：数据库写入失败、查询超时、参数非法等场景？
□ 验收标准是否每条都可测试？
```

Step 3：确认无误后，生成任务清单并执行

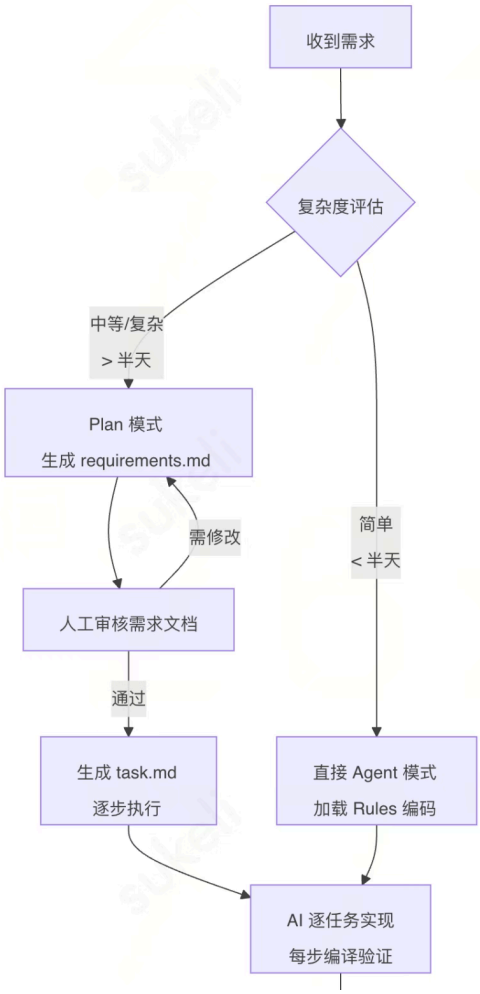
```
需求审核通过。请阅读 .codebuddy/plan/feat-operation-log/requirements.md，
生成 .codebuddy/plan/feat-operation-log/task.md 任务清单，
然后按任务顺序逐步实施。每完成一个任务后自动编译验证。
```

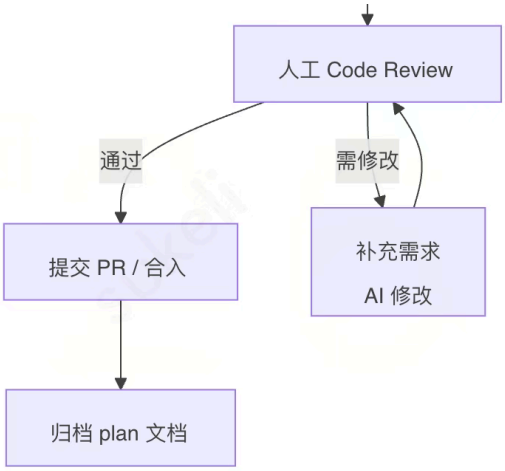
Step 4：人工审查代码并归档

```
# 归档
mv .codebuddy/plan/feat-operation-log .codebuddy/plan/archive/feat-operation-log
```

六、日常开发 SOP（标准操作手册）

6.1 SOP-A：新需求开发



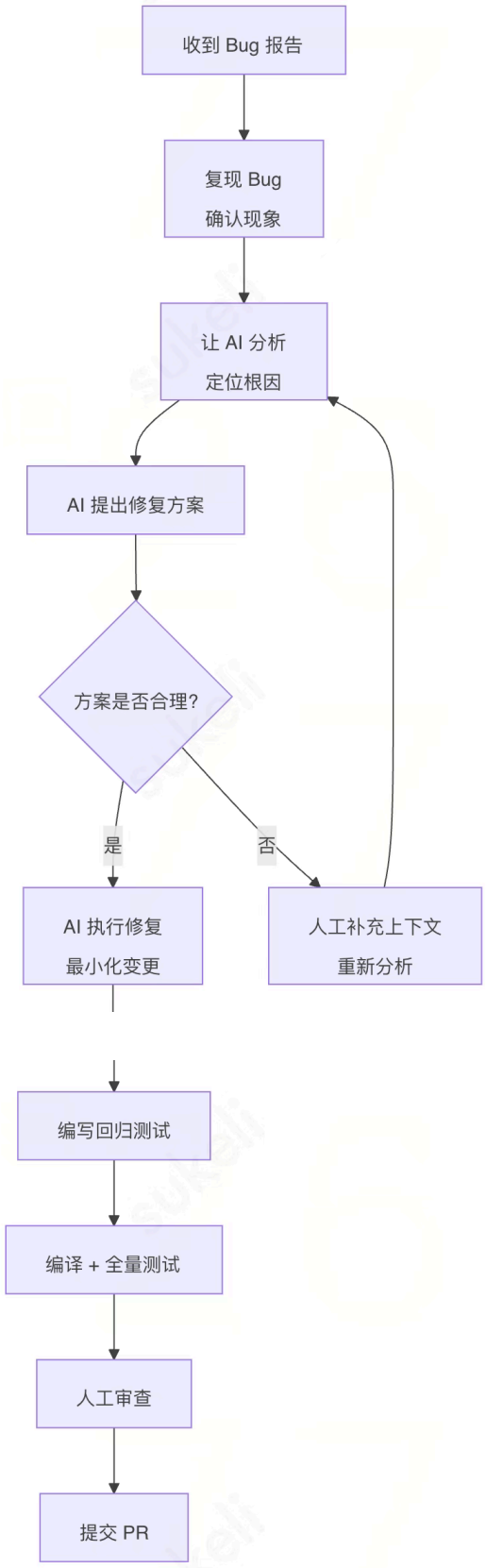


简单需求的快捷流程（< 半天工作量）：

```
# 直接在 Agent 模式中描述需求，无需生成 requirements.md
# 但仍需遵守 Rules 约束

请在 internal/user/service.go 中新增一个 GetUserProfile 方法，
要求：
1. 通过 user_id 查询用户基本信息
2. 返回 UserProfileResponse 结构体
3. 包含错误处理和日志记录
4. 编写对应的单元测试
```

6.2 SOP-B：Bug 修复



Bug 修复红线：

- 一个 PR 只修一个 Bug，禁止夹带其他修改
- 必须编写能复现该 Bug 的测试用例
- commit 信息格式：`fix: [模块名] 修复xxx问题 (#issue编号)`

6.3 SOP-C：AI 辅助 Code Review

方式一：提交前自查

```
请对以下文件的变更进行 Code Review：
- internal/user/service.go
- internal/user/repository.go

审查要点：
1. 是否符合分层架构规范
2. 错误处理是否完善
3. 是否有潜在的性能问题
4. 命名是否规范，注释是否清晰
5. 是否有安全隐患
```

方式二：Review 他人 PR

```
请阅读以下 PR 的变更内容，给出 Code Review 意见：
[粘贴 diff 或指定文件列表]

重点关注：逻辑正确性、边界情况处理、与现有代码的一致性、测试覆盖度
```

Code Review 检查清单：

类别	检查项	说明
架构	分层是否正确	Controller 不含业务逻辑，Repository 不含业务判断
架构	是否复用已有模块	检查 `pkg/` 下是否有可复用的工具
质量	错误处理	所有 error 必须显式处理，禁止 `_ = err`
质量	单元测试	核心逻辑必须有测试，覆盖正常和异常路径
安全	SQL 注入	参数化查询，禁止字符串拼接 SQL
安全	敏感信息	禁止硬编码密钥、连接串
性能	数据库查询	检查是否有 N+1 查询、全表扫描
规范	commit 信息	格式清晰，描述修改点和原因

七、团队协作红线（不可违反）

红线	说明
先 Spec 后 Code	严禁在没有明确 Spec 的情况下直接开始 Coding
Rules 共享	项目级的 Rules 必须同步至 Git 仓库，不允许本地私有
Skill 沉淀	通用的逻辑处理必须抽象为 Skill 以便全队复用
MCP 优先	关键元数据优先通过 MCP 实时同步，而不是手动维护副本
变更可追溯	所有代码变更必须附带清晰的 commit 信息

八、常见问题与反模式

8.1 反模式清单

#	反模式	现象	正确做法
1	巨型 Prompt	一次性把几千字需求丢给 AI	先 Plan 模式生成 requirements.md，拆解后逐步执行
2	跳过审核直接编码	觉得需求简单，不写 Spec 直接让 AI 写代码	半天以上的需求必须走 Plan 模式
3	Rules 写了不维护	Rules 文件写完就放着，半年后已经和实际规范脱节	月度 Review 会议定期检查
4	MCP 过度接入	接入了十几个 MCP Server，Token 消耗暴增	只接入 P0/P1 优先级的 MCP
5	Skill 不原子化	一个 Skill 塞了太多功能	一个 Skill 只解决一类问题
6	盲目信任 AI 输出	AI 生成的代码不审查直接合入	所有 AI 生成的代码必须经过人工 Code Review
7	Chat 历史当文档	需求细节全在聊天记录里	需求和设计决策必须持久化到 `.codebuddy/plan/`
8	一个 PR 改所有	让 AI 一次性实现多个不相关的功能	一个 PR 只解决一个问题

九、合规性自检：用 自制 Skill 一键体检

前面几章把规范、工具、SOP 都讲清楚了。但落地最大的痛点是——**规范写完容易，执行下去难**。团队成员是否真的按规范配置了 Rules？项目是否真的建了 `.codebuddy/skills/`？Commit 信息是否规范？

靠人工一个项目一个项目地翻，又慢又容易漏。我们根据上面的规范沉淀了 `harness-audit` Skill —— 把整套规范的检查项固化成一个可执行的合规性审计工具，**一句话就能给项目打分、找问题、给建议**。

9.1 这个 Skill 能干什么

`harness-audit` 是一个 Harness 规范的自动化合规性检查工具，覆盖前文规范涉及的所有核心维度。它能做三件事：

1. **打分**：给项目从 7 个维度打总分（满分 100），按 S/A/B/C/D 五级评定
2. **诊断**：列出每个维度的具体问题（哪些缺失、哪些不规范、哪些有但没用好）
3. **开方**：按 P0/P1/P2/P3 优先级给出改进建议，附带操作步骤和代码示例

与本规范的对应关系：

审计维度	权重	对应章节	检查内容
1. AGENTS.md（AI 说明书）	15%	§4.4 编写 AGENTS.md	是否存在、是否精简（~100 行）、是否包含项目概述/架构/目录/常用命令

审计维度	权重	对应章节	检查内容
2. Rules（约束体系）	20%	§4.3 Rules 配置	`.codebuddy/rules/` 目录、Frontmatter 规范、架构/风格/安全约束完整性
3. Skills（技能沉淀）	15%	§5.4 Skills 配置	`.codebuddy/skills/` 目录、Skill 数量、SKILL.md 规范性、业务相关性
4. MCP（上下文扩展）	10%	§5.1 MCP 配置	`mcp.json` 是否存在、Server 配置规范、敏感信息是否硬编码
5. Plan 模式（SDD）	15%	§5.5 Spec 与 Plan 模式	`.codebuddy/plan/` 目录、requirements.md / task.md 完整性
6. 项目工程规范	15%	§6 日常开发 SOP	目录结构、分层架构、README、依赖管理、`.gitignore`
7. Commit 规范与协作	10%	§6.2 Bug 修复红线 / §7 团队协作红线	Commit 格式（type: [scope] description）、变更粒度

可以看到，每个审计维度都精确对应到本规范的某一章节——**Skill 就是规范的可执行版本**。

9.2 怎么用

前置条件：

- 已安装 CodeBuddy 插件并完成基础配置（参见 §4.1）
- 已将 `harness-audit` Skill 放到 `.codebuddy/skills/` 目录下，或通过团队 Skills 仓库同步

触发方式（在 CodeBuddy Agent 模式下输入即可）：

```
# 审计当前本地项目
请用 harness-audit Skill 对当前项目做一次合规性审计。

# 审计远程工蜂项目（需配合工蜂 MCP）
请用 harness-audit Skill 审计这个项目：
https://git.xxx.com

# 只关注某几个维度
请用 harness-audit Skill 审计当前项目，重点检查 Rules 和 Skills 维度。
```

AI 会自动加载 Skill，按"信息采集 → 逐维度评分 → 生成报告"三阶段执行，最终把完整报告写入 `.codebuddy/reports/harness-audit-{项目名}-{日期}.md`，并在对话中展示摘要。

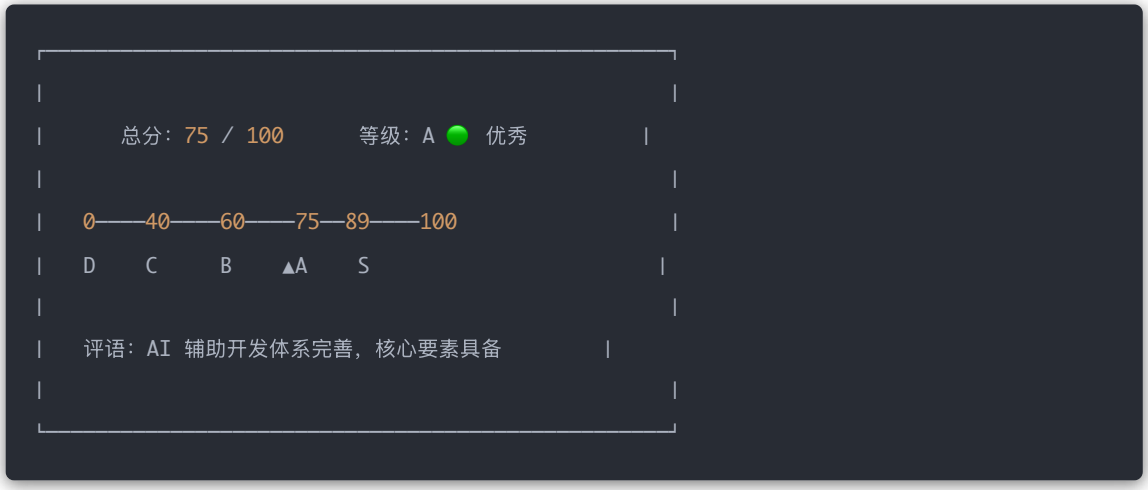
9.3 检查结果示例

下面是对一个真实 Go 后端项目执行审计后的报告摘要（节选自完整报告）：

📋 项目基本信息

项目	信息
项目名称	go_scaffolding_svr
项目地址	git.xxx.com
项目负责人	zhangsan（基于 Git 提交记录分析）
审计分支	master
技术栈	Go 1.21 + tRPC-Go
最近活跃	2026-04-15 18:32
Commit 总数	287
核心贡献者	zhangsan (158)、lisi (72)、wangwu (35)

🎯 总体评分



维度	得分	满分	得分率	等级
AGENTS.md	14	15	93%	🟢 优秀
Rules	18	20	90%	🟢 优秀
Skills	12	15	80%	🟢 优秀
MCP	0	10	0%	🔴 不合格
Plan 模式	12	15	80%	🟢 优秀
工程规范	13	15	87%	🟢 优秀
Commit 规范	6	10	60%	🟡 良好
总计	**75**	**100**	**75%**	🟢 **A 级**

Harness 合规性各维度得分率 AGENTS.mdRulesSkillsMCPPlan 工程 Commit1009080706050403020100得分率 (%)

✅ 亮点

- **AGENTS.md 高度精简**：78 行，符合"目录索引而非百科全书"的设计原则

- **Rules 体系完善**：`.codebuddy/rules/` 下有 `global.md`、`go-backend.md`、`security.md` 三个文件，覆盖架构、风格、安全
- **Skills 业务相关性强**：沉淀了 `rainbow-config`、`polaris-resource` 等 5 个业务 Skill
- **目录结构标准**：严格遵循 `cmd/internal/pkg/api` 标准布局

⚠️ **主要问题**

- 🔴 **未配置 MCP**：项目根目录无 `mcp.json`，AI 无法实时读取数据库 Schema 和工蜂 Issue
- 🟠 **Commit 信息不规范**：最近 50 条 commit 中，30% 使用 "update"、"fix bug" 等模糊描述
- 🟡 **Plan 目录未启用归档**：`.codebuddy/plan/` 下没有 `archive/` 子目录，已完成需求未归档

🔧 **优化建议（节选）**

🔴 **P0 - 立即修复**

1. **接入 DB MCP**（耗时 30 分钟，参见 §5.1.2）

```
{
  "mcpServers": {
    "db-mysql": {
      "command": "npx",
      "args": ["-y", "@anthropic/mcp-server-mysql"],
      "env": {
        "MYSQL_HOST": "${DB_HOST}",
        "MYSQL_USER": "readonly_user",
        "MYSQL_PASSWORD": "${DB_PASSWORD}"
      },
      "timeout": 10000,
      "transportType": "stdio"
    }
  }
}
```

🟠 **P1 - 短期改进**

1. **统一 Commit 规范**（耗时 1 周）：在团队周会同步 §6.2 Commit 格式（`type: [scope] description`），并配置 git hook 自动校验
2. **建立 Plan 归档机制**（耗时 30 分钟）：`mkdir -p .codebuddy/plan/archive`，已完成需求统一归档

🚀 **Quick Wins**

改进项	预计耗时	影响
创建 `mcp.json` 接入 DB MCP	30 分钟	AI 写 SQL 准确率提升 30%+
创建 `.codebuddy/plan/archive/`	5 分钟	历史需求可追溯

改进项	预计耗时	影响
配置 commit-msg hook	20 分钟	Commit 规范率从 70% → 95%+

📈 **成熟度路线图**



> **当前阶段**：第二阶段（工具接入）

> **下一阶段目标**：补齐 MCP 接入、规范 Commit、建立归档机制 **预计达成时间**：2 周

> 完整报告（含 7 个维度的逐项检查表、Mermaid 饼图、落地手册对标检查等）会输出到 `.codebuddy/reports/harness-audit-go_scaffolding_svr-20260416.md` 。

9.4 推荐使用节奏

场景	频率	用途
项目初次接入规范	1 次	摸清基线，定改进计划
季度团队复盘	每季度 1 次	量化规范落地效果，对比上季度
新项目立项后	立项 2 周内	检查基础建设阶段是否到位
Code Review 之前	按需	配合 §6.3 SOP-C，做提交前自查
Knot 平台共享审计	每月 1 次	跨项目对比，识别 S 级标杆项目

> ⚠️ **注意**：审计报告是体检结果，不是 KPI。重点是发现问题、推动改进，不要把分数当指标考核。规范的目的永远是让 AI 更好用、让团队效率更高，而不是为了刷分。

第三部分：总结

十、总结

> Django 创始人说过：交付代码的成本已经接近免费了，但交付好代码的成本依然很高。

AI Agent 工具能在代码质量的各个方面帮不少忙，但最终的质量把关，还是得靠操作这些工具的人。你得知道什么是好代码，你得能判断 Agent 产出的东西够不够好，你得能在关键的地方做出正确的取舍。

成本降低了，标准不能降低。工具变强了，人的判断力要跟着变强。

```
让各类工具适配规范，
而不是靠个人去适配各类工具。
这就是从"人驱动 AI"到"AI 自驱动"的转变。
```

通过这套规范体系，团队可以：

- 把"交付代码"的成本降低（AI 执行）
- 把"交付好代码"的标准写进 Harness 系统里（规范约束）
- 实现"知识飞轮效应"——新成员越多，整体效率反而越高（经验沉淀）

要掌握底层生存法则：流水的工具，铁打的规范。

如有不足,欢迎讨论指正。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 5.0万

留言

写留言

还没有留言，来写首评